# Table of Contents


**1. Introduction**
- What this is
- Goals vs content
- Architecture sketch
- Main pieces
- Stack
- Layout of the repo
- Setup and runtime notes
  - Engine
  - Plugins and modules
  - Maps and config worth knowing
  - Building
  - Running
  - Typical problems

**2. System Architecture**
- How the pieces hang together
- Agents and flocking
- Schedules
- Needs, priority, fallback
- Stations
- Navigation
- Leadership and crime (optional)
- Evacuation
- Analytics and validation
- Feature registry

**3. XML Schedule System**
- Where the files are
- Time
- Skeleton
- Actions
- Rules (preferred)
- Older task tags
- Conditions
- MoveTo, Idle, Trigger
- UseStation
- Practical notes

**4. Use Cases**
- Use Case 1 — Emergency Evacuation
- Use Case 2 — Daily Urban Life
- Use Case 3 — Urban Infrastructure Planning

**5. Validation and Verification**
- Common validation setup
- V01 — Unidirectional
- V02 — Bidirectional
- V03 — Corner

V04 — Bottleneck
V05 — T-junction
V06 — Functional area
Directional schedule templates
Running a validation map
Mall A/B infrastructure verification
Debug panel
Validation Statistics
Panel fields
Reading a run
Other debug controls

**6. Configuration**
SimulationConfig.xml
Feature flags
Daily Urban Life
Evacuation
Infrastructure planning
Leadership
Cops and Robbers
Validation Statistics
AFlockController properties
Schedule manager
Stations and areas
Navigation
Analytics files

**7. Core Classes**
ABoidsAB
AFlockController
AScheduleManager
AAgentStation and AAgentArea
Layered agent components
Emergency and world actors
Use-case managers
Simulation subsystems and configuration
Navigation helpers
Analytics data
Legacy and peripheral code

**8. Extending the Project**
Adding a map
Writing a schedule
Adding stations
Building a validation scenario
Extending agent behaviour
Adding a use case
Extending the Mall comparison

**9. Known Limitations**
Content and maps
Schedules
Legacy leftovers

Navigation
Stations
Per use case
Validation
Defaults that surprise people
Performance

**10. Conclusion**

# 1. Introduction

## What this is

Pedestrian crowd sim in Unreal Engine 5.4. Agents are `ACharacter` (`ABoidsAB`) — local flocking plus NavMesh path following. Day-to-day routines come from XML schedules. Stations (desks, counters, beds, etc.) have capacity and queues. Three scenarios sit on that stack: emergency evacuation, daily urban life, and a mall A/B layout comparison. Smaller validation maps (`V01`–`V06`) are there for simpler flow checks.

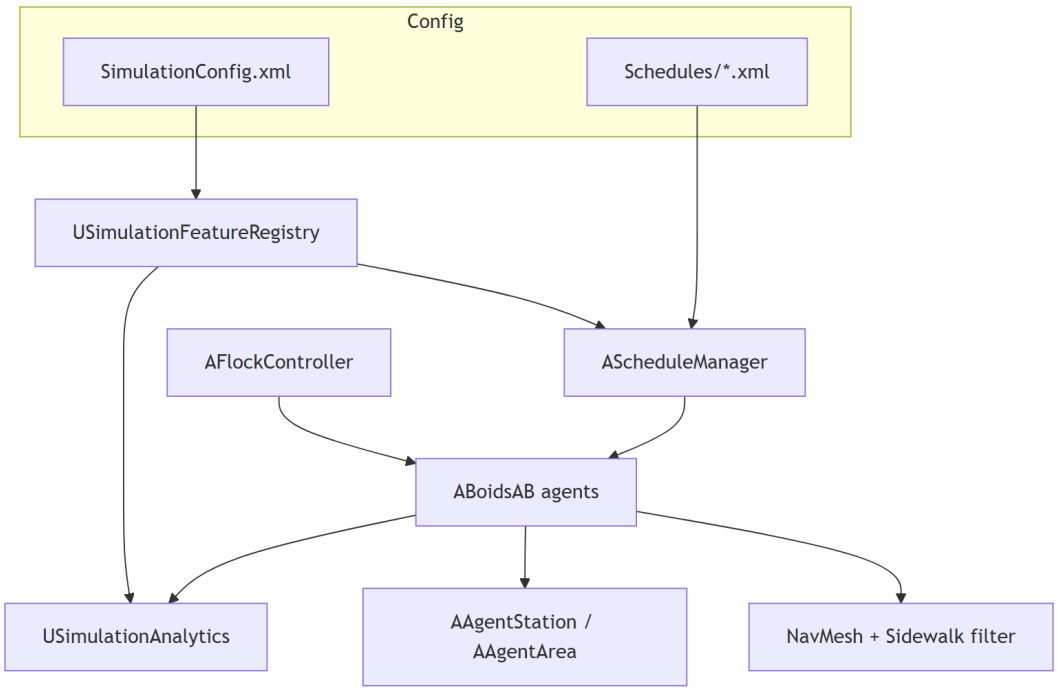
Logic lives in the `Thesis` C++ module. Feature toggles are in `Config/SimulationConfig.xml`, applied at runtime by `USimulationFeatureRegistry`.

## Goals vs content

What we were aiming for, and where it ended up:

| Aim | In the project |
|---|---|
| Evacuation under time pressure | `Evacuation.umap`, `AEmergencyManager`, exits, CSV |
| Authorable daily routines | `DailyUrbanLife.umap`, schedule XML, homes/desks |
| Compare two mall layouts | `MallA` / `MallB`, infrastructure + V&V export |
| Local crowd motion | `ABoidsAB`, `AFlockController` |
| Goal-directed paths | NavMesh + sidewalk filter |
| Simple flow experiments | `V01`–`V06`, on-screen stats panel |

## Architecture sketch



Architecture diagram 1

Most gameplay logic ends up on the agent. The registry decides which managers and subsystems are allowed to run for the current map. Schedules push commands; flocking and navigation shape how those commands look in motion.

### Main pieces

| Area | Main types | Notes |
|---|---|---|
| Agents | `ABoidsAB`, `AFlockController` | Motion and flock parameters |
| Schedules | `AScheduleManager` | Clock + XML rules |
| Stations | `AAgentStation`, `AAgentArea` | Capacity and queues |
| Decision layers | Needs, priority, fallback components | Hunger/energy and conflict handling |
| Navigation | NavMesh, sidewalk area/filter | Path queries |
| UC1 | Emergency / incident / exits / routing | Evacuation |
| UC2 | `ADailyUrbanLifeManager` | Destinations, offsets, speeds |
| UC3 | Infrastructure + V&V managers | Mall metrics and comparison |
| Optional demos | Leadership, cops/robbers | Off in the baseline config |
| Metrics | `USimulationAnalytics`, validation panel | CSV and on-screen stats |

### Stack

Unreal Engine **5.4**, C++ (`Thesis` module), `XmlParser`, `NavigationSystem`, `AIModule`, `EnhancedInput`, UMG (widgets such as `WBP_Evacuation` / `WBP_DailyLife`). The `.uproject` also enables ModelingToolsEditorMode for editor-only work.

### Layout of the repo

```
Thesis/
├── Config/                SimulationConfig.xml, engine ini
├── Content/UseCases/      Evacuation, DailyUrbanLife, MallA/B, Validation
├── Documentation/         This guide set
├── Schedules/             XML itineraries (also copied under Saved/)
├── Source/Thesis/         C++ module (AI, Emergency, Simulation, …)
└── Thesis.uproject
```

Exported CSVs go under `Saved/Analytics/` after a run that actually triggers export.

## Setup and runtime notes

### Engine

Open the project with **Unreal Engine 5.4** (`Thesis.uproject`). The runtime module is `Thesis`. These notes assume Win64 Development Editor.

### Plugins and modules

The only plugin listed in the `.uproject` is `ModelingToolsEditorMode` (editor targets). Module dependencies in `Thesis.Build.cs` include Core, Engine, XmlParser, NavigationSystem, AIModule, EnhancedInput, and related engine modules. UMG is pulled in for the use-case widgets.

### Maps and config worth knowing

From `DefaultEngine.ini`:

- Editor startup map: `/Game/UseCases/Validation/V04_Bottleneck`
- Game default map: `/Game/ScheduleTest/Untitled`

For demos, open the use-case map you care about from the Content Browser rather than relying on the game default.

Simulation features are in `Config/SimulationConfig.xml`. At runtime the game prefers `Saved/Config/SimulationConfig.xml` and will copy the project file there the first time Saved is missing. Edit Saved for local experiments.

### Building

Generate project files from the `.uproject` if needed, then build **ThesisEditor** (Development Editor, Win64).

Command-line example:

```
"C:\Program Files\Epic Games\UE_5.4\Engine\Build\BatchFiles\Build.bat" ThesisEditor Win64
        Development -Project="…\Thesis\Thesis.uproject" -WaitMutex
```

If the editor holds Live Coding / UBT, an external build can sit on the mutex until PIE or Live Coding releases it. Compiling inside the editor is often easier while iterating.

### Running

1. Open a use-case or validation map.
2. Confirm the level has the actors that scenario needs (ScheduleManager, FlockController, agents, stations, managers). Placement is in the `.umap`.
3. Play (PIE) or standalone.

Useful maps:

| Scenario | Asset |
|---|---|
| Evacuation | `/Game/UseCases/Evacuation` |
| Daily urban life | `/Game/UseCases/DailyUrbanLife` |
| Mall A / B | `/Game/UseCases/UrbanPlanning/MallA` or `MallB` |
| Validation | `/Game/UseCases/Validation/V0*_…` |

Map-name gates flip some features automatically (Daily Urban Life, malls, validation). Schedules are XML under `Schedules/` (runtime often uses the copy under `Saved/Schedules/`). Rebuild NavMesh after layout changes; sidewalk preference needs sidewalk areas consistent with `UNavArea_Sidewalk`.

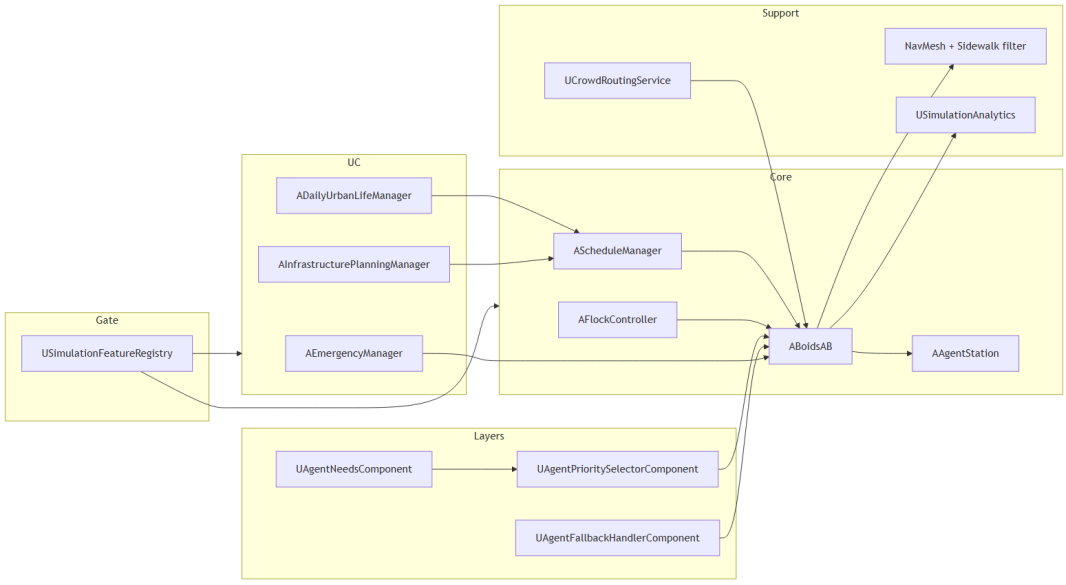
### Typical problems

| What you see | Usual cause | What to check |
|---|---|---|
| Schedules ignored | Wrong path, IDs mismatch, agent not registered | ScheduleManager path vs `Agent_N` IDs |
| No path / empty waypoints | Off-mesh start/goal or filter | NavMesh, sidewalk connectivity |
| Station never starts | Arrival XY/Z not met | Path actually reaches station root |
| Wrong features | Config vs map gate | `Saved/Config/SimulationConfig.xml` |
| Build hangs | Mutex / Live Coding | Close PIE or compile in-editor |
| RuleParser path spam | Legacy absolute paths | Use ScheduleManager schedules |
| Empty Analytics folder | Export never ran | Finish a UC1/UC3 session with auto-export |
| No validation panel | Feature / CVar off | Validation map + `thesis.ValidationDebugPanel 1` |

# 2. System Architecture

C++ side of the runtime. Blueprint wiring and actor placement live in the binary maps—open the level in the editor if you need exact placement.

### How the pieces hang together




Architecture diagram 2

### Agents and flocking

`ABoidsAB` is the pedestrian. `AFlockController` holds shared weights and speeds; it does not tick the crowd itself.

In normal play an agent registers with the flock, steers with separation / alignment / cohesion / avoidance, and follows waypoint lists from NavMesh queries. While MoveTo is active, flock forces can be blended in as a **lateral** offset on path progress (`bEnableSteeringWhileMoving`) so the corridor does not get abandoned for pure flocking.

States on the agent include `Normal`, `Fleeing`, `Exiting`, `MovingToSafeZone`, `EvacuationSuccess`, and `Casualty`.

Waypoint advance uses both horizontal distance and feet height against the path point. That matters on stairs and multi-floor meshes: XY-only advance would skip elevated samples. Look-ahead along a segment is done in the horizontal plane so steep chords do not reverse agents back up a stair.

### Schedules

`AScheduleManager` owns the sim clock, loads XML, registers agents by id (`Agent_0`, …), and fires rules when due. Station occupation can defer a rule. Targets for MoveTo / UseStation are resolved through actor tags and areas. Spawners can assign per-agent schedule overrides.

Clock detail used in UC2 comments: about one real second per schedule minute.

There is still an older `ARuleParser` in the tree with hard-coded external paths. Active schedules for the use cases go through ScheduleManager.

### Needs, priority, fallback

`UAgentNeedsComponent` tracks hunger (up) and energy (down) with critical thresholds. `UAgentPrioritySelectorComponent` orders layers from panic/emergency down through needs, crime, leadership, scheduled tasks, then idle flock. `UAgentFallbackHandlerComponent` reacts to bad paths, stuck agents, queues, and missing destinations (retry, alternate POI, wait, skip, idle, go home, and similar).

When a higher layer owns the agent, schedules can be deferred (for example while following a leader).

### Stations

`AAgentArea` groups stations and can be marked closed. `AAgentStation` has capacity, a FIFO queue, timed use after the occupant has physically arrived, and optional animation / pose settings.

`UseStation` from XML can be one step or a pipeline (counter then table, for example). Timing starts on `NotifyOccupantReached`, not merely when a path is requested.

### Navigation

Paths come from `FindPathToLocationSynchronously` with `UNavFilter_SidewalkOnly`. Sidewalk areas (`UNavArea_Sidewalk`) are cheap; other areas are heavily penalised. Agents follow the resulting points with Character Movement. Optional NavMesh snap corrects XY if the agent drifts off mesh.

Evacuation exit choice scores distance and congestion via `UCrowdRoutingService` and `UCongestionAnalyzer`. Other routing modes exist on the enum but evacuation is the path that is fully wired for exit selection.

### Leadership and crime (optional)

Leadership: `UAgentLeadershipComponent` plus `ALeadershipDemoManager` when the feature is on. Followers can override schedules if configured.

Cops and robbers: `UAgentCrimeRoleComponent` and `ACopsAndRobbersDemoManager`. Baseline config keeps this feature off.

### Evacuation

`AEmergencyManager` and `AIncidentManager` start and end the global phase. `AExitPoint` handles capacity and queue slots. `AEvacuationSafeZone` is a tagged safe destination. Agents flee after a reaction delay, may switch exits under congestion rules, and may become casualties if that option is enabled. Metrics export covers evacuations, casualties, reassignments, and per-exit throughput.

### Analytics and validation

`USimulationAnalytics` holds snapshots and CSV writers for evacuation, daily life, infrastructure, crime, and validation sampling. Daily-life export exists as an API but the UC2 manager does not call it automatically at session end. Validation has a live panel and no CSV writer. Mall A vs B comparison files are produced by `AVerificationValidationManager` when both infrastructure CSVs are available.

### Feature registry

`USimulationFeatureRegistry` is the gate for Evacuation, Leadership, CopsAndRobbers, InfrastructureAnalysis, DailyUrbanLife, and ValidationStats. After config load, map-name gates adjust those flags (malls turn infrastructure on and daily life off; validation maps turn validation stats on).

| Use case | Managers | Typical schedule | Metrics |
|---|---|---|---|
| UC1 | Emergency + Incident | Level setup | Evacuation CSV |
| UC2 | DailyUrbanLifeManager | `DailyUrbanLifeSchedule.xml` | Live snapshot |
| UC3 | Infrastructure (+ V&V) | Mall A/B XML | Infrastructure CSV + comparison |
| Validation | Map gate + panel | `Schedules/Validation/*` | On-screen only |

## 3. XML Schedule System

Schedules are plain XML files read by `AScheduleManager`. Only the actions and attributes below are actually parsed. Simulation feature toggles live in `SimulationConfig.xml`, not in these schedule files.

### Where the files are

| Path | Used for |
|---|---|
| `Schedules/DailySchedule.xml` | Older / generic day |
| `Schedules/DailyUrbanLifeSchedule.xml` | UC2 |
| `Schedules/InfrastructurePlanningSchedule_MallA.xml` | UC3 Mall A |
| `Schedules/InfrastructurePlanningSchedule_MallB.xml` | UC3 Mall B |
| `Schedules/MultiStageStationExample.xml` | Multi-stage UseStation sample |
| `Schedules/Validation/*.xml` | Validation maps and left/right spawner templates |

ScheduleManager's `RelativeSchedulePath` (or a manager override) chooses the file. Under PIE you often end up on the copy in `Saved/Schedules/`.

### Time

Times are `HH:MM` on the schedule clock. UC2 comments treat one schedule minute as about one real second. Rules can be dropped as stale after the retry window (default 45 schedule minutes).

### Skeleton

```
<?xml version="1.0" encoding="UTF-8"?>
<schedule>
  <agent id="Agent_0" role="Civilian">
    <!-- rules and/or tasks -->
  </agent>
</schedule>
```

`id` must match the registered agent. Optional `role` (`Police` / `Robber` / `Civilian`) is applied on register when cops-and-robbers is enabled.

### Actions

| `action` | Behaviour |
|---|---|
| `MoveTo` | Path to first actor with the given area/tag |
| `Idle` | Idle for `duration` seconds |
| `Trigger` | `OnScheduleTrigger` with the `id` string |
| `UseStation` | Occupy a station (one step or a pipeline) |
| `None` | No-op |

There is no `Break` action. UC2 breaks look like:

```
<then action="Trigger" id="Break" />
```

`Break` / `Pause` are special-cased so station blocking does not trap them the same way as other rules.

### Rules (preferred)

```
<agent id="Agent_0">
  <rule time="00:22" prob="1.0">
    <if condition="location == Work"/>
    <then action="UseStation" area="Work0" stationType="A0" duration="15"/>
    <else action="MoveTo" area="Work0"/>
  </rule>

  <rule time="00:40">
    <then action="MoveTo" area="Home1"/>
  </rule>
</agent>
```

Optional `prob` on the rule. Nested `<else><then …/></else>` is accepted.

### Older task tags

Still loaded if present:

```
<task time="00:30" action="MoveTo" target="ShopA" id="" prob="1.0" duration="-1" stationType=""/>
```

Prefer `<rule>` for new work.

### Conditions

Only:

- `emergency == true` / `emergency == false`
- `location == Home` / `Work` / `Food` (within `LocationCheckRadius`, default 200 cm)

Anything else evaluates false and logs a warning.

### MoveTo, Idle, Trigger

```
<then action="MoveTo" area="Exit_Right"/>
<then action="Idle" duration="8"/>
<then action="Trigger" id="Break"/>
```

### UseStation

Single step on the `<then>`:

```
<then action="UseStation" area="ShopA" stationType="ShopA_Desk" duration="10"/>
```

`area` is the area tag. `stationType` matches `StationIdentifier` / station tag. `duration` in seconds (−1 → station default). Optional `station` on a **child** stage binds an explicit actor name.

Multi-stage:

```
<then action="UseStation">
  <stage area="FoodCourt" stationType="FoodCounter" duration="10"/>
  <stage area="FoodCourt" stationType="FoodTable" duration="14"/>
</then>
```

Child tags may be `stage`, `step`, or `interaction`. See `MultiStageStationExample.xml`.

### Practical notes

Agents must arrive at the station (XY and height checks) before interaction time runs. Full stations queue. Closed areas affect availability. Mall food flows rely heavily on multi-stage UseStation.

# 4. Use Cases

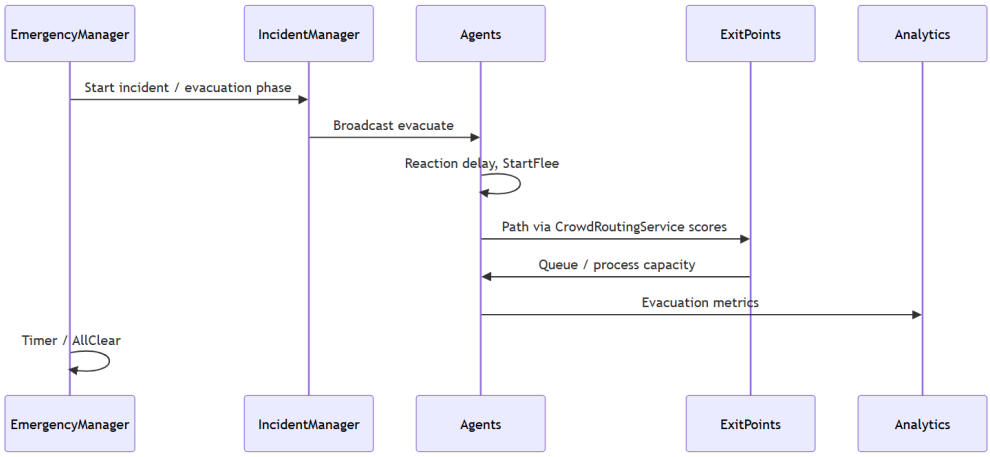
Screenshots were not available in the repository, so the use-case write-ups refer to the maps and schedules by name only (`Evacuation.umap`, `DailyUrbanLife.umap`, `MallA.umap`, `MallB.umap`).

Three scenarios sit on the same agent stack. Which managers run is controlled by `USimulationFeatureRegistry` and `SimulationConfig.xml`, with some maps flipping flags by name.

### Use Case 1 — Emergency Evacuation

The evacuation map (`Content/UseCases/Evacuation.umap`) is meant to show a timed emergency: agents flee, pick exits with distance and congestion in mind, queue at capacity-limited exits, and may reach safe zones or become casualties.

Unlike the daily-life and mall maps, evacuation is **not** turned on just by the map name. `enableEvacuation` must be true in config (it is in the project baseline), and the level needs `AEmergencyManager`, `AIncidentManager`, exits, agents, and usually a flock controller. Safe zones are optional.




Architecture diagram 3

Rough flow: the manager starts the phase; agents wait out their reaction delay, then flee; exit scores mix distance and congestion; agents can be reassigned a limited number of times; exits process queue slots; the timer and casualty options follow the `<evacuation>` block in config. A finished session can write `Saved/Analytics/Evacuation_<timestamp>.csv`.

Tunables include radius, weights, reevaluation interval, switch threshold, max reassignments, reaction delay range, duration, timer start delay, speed variation, and casualty / celebration flags. Any on-screen controls in `WBP_Evacuation` are Blueprint-side.

In play you should see schedules stop for evacuating agents, flee/exit motion instead of idle flocking, and analytics reflecting throughput, queues, timer state, and reassignments when congestion actually triggers switches.

### Use Case 2 — Daily Urban Life

Map: `DailyUrbanLife.umap`. Schedule: `DailyUrbanLifeSchedule.xml` (path configurable). The DailyUrbanLife feature is off in the clean baseline config but turns on automatically when the map name contains `DailyUrban`, `UrbanLife`, or `DailyLife`.

`ADailyUrbanLifeManager` picks the schedule, wires home/work/food from tags, applies small minute offsets and walking-speed ranges, warns if expected tags are missing, and starts analytics sampling.

The authored day is compressed: **home** → **commute** → **work desk** → **break** → **work** → **home** (and in the current schedule, some agents also use Sleep/Bed stations). Work areas are `Work0`–`Work6` with desks `A0`–`A24`. Homes are `Home1`–`Home25`. Agents 0–24 own desks; 25–49 reuse them and will queue. Breaks use `Trigger` with `id="Break"` toward a plaza tag (default `Plaza`).

If needs are enabled, hunger and energy move over time (scaled by `needsRateScale`). Critical needs can interrupt through the priority layer; scheduled home return can still win when `homeReturnOverridesNeeds` is set.

Leadership is a separate optional feature and is not required for the baseline day schedule. Casual greetings from `UPedestrianInteractionComponent` may pause agents briefly.

Worth checking in a run: staggered starts, queues on shared desks, break behaviour (plaza or in-place fallback), and snapshot fields on analytics. Daily-life CSV export exists on the analytics subsystem but is not fired automatically when the UC2 manager ends a session.

### Use Case 3 — Urban Infrastructure Planning

This use case compares two mall layouts with the same pedestrian code.

| Layout | Map | Schedule | Auto service capacity |
|---|---|---|---|
| Mall A (tighter food service) | `MallA.umap` | `InfrastructurePlanningSchedule_MallA.xml` | 1 |
| Mall B (two food courts) | `MallB.umap` | `InfrastructurePlanningSchedule_MallB.xml` | 2 |

Opening a mall map forces infrastructure analysis on and daily urban life off. With `autoSelectFromMap`, schedule and capacity follow A vs B. Mall A comments describe one food court and counter→table pipelines; Mall B splits load across `FoodCourt` and `FoodCourt2`.

Agents shop, run multi-stage food service, sit, and exit. Queues and NavMesh paths (stairs included) are where layout differences show up. At the end of a run, infrastructure CSVs land under `Saved/Analytics/`. When both A and B CSVs exist, the V&V manager can write a comparison CSV and a short text summary with checklist lines (metrics present, wait/congestion hypotheses, and so on).

Numbers always depend on how many agents you spawn and how the mesh and stations were placed in each map. Empirically, Mall A tends to pressure shared counters harder; Mall B spreads that load—confirm on your own runs rather than treating the hypothesis text as a unit test.

# 5. Validation and Verification

Validation uses authored Unreal maps, XML schedules, and live analytics. The project has no Unreal Automation or Functional Test C++ classes, and the validation statistics are not exported to CSV.

The available pedestrian-flow maps are `V01` to `V06` in `Content/UseCases/Validation/`. Separate crosswalk, merge, and split maps do not exist. There is also no separate corridor map. Names matching `Validation` or `V01_`–`V09_` are recognised by the map-driven feature gates, although the project contains no V07, V08, or V09 maps.

### Common validation setup

Each map uses a schedule from `Schedules/Validation/`. Agents are normally identified as `Agent_0`, `Agent_1`, and so on, matching the IDs in the XML. Depending on the map, agents may be placed directly in the level or created by `ADirectionalSpawner`. The spawner can assign per-agent schedule overrides using `Validation_MoveLeft.xml` and `Validation_MoveRight.xml` as templates.

Time comes from the same `AScheduleManager` simulation clock used by the other use cases. `USimulationAnalytics` samples an `FValidationAnalyticsSnapshot` for the on-screen panel. The snapshot contains the scenario name, simulation time, spawned, active, completed, and failed agent counts, travel-time statistics, average speed, average path length, throughput, flow rate, queue length, density, waiting time, congestion time, FPS, moving and waiting counts, and current average velocity.

Two fields use negative sentinel values. `AgentsFailed` is `-1` because failed-agent tracking is not supported, and `AveragePathLength` is `-1` because path length is not tracked. The panel renders these values as `n/a`.

### V01 — Unidirectional

`V01_Unidirectional.umap` uses `Validation_Unidirectional.xml`. Agents idle briefly and then move to the actor tagged `Exit_Right`. The run should form a one-way stream, with the completion count and throughput increasing as agents reach the exit. Check that `Exit_Right` resolves correctly and that opposing collisions are uncommon.

### V02 — Bidirectional

`V02_Bidirectional.umap` uses `Validation_Bidirectional.xml`. According to the schedule comments, `Agent_0`–`Agent_9` move to `Exit_Right`, while `Agent_10`–`Agent_19` move to `Exit_Left`.

The two streams share the same space. Under comparable density, this should produce more interference, waiting, and congestion than V01. Confirm that both exit tags are used and watch the waiting and congestion values where paths conflict.

### V03 — Corner

`V03_Corner.umap` uses `Validation_Corner.xml`. Every agent moves to `CornerExit`. The NavMesh route bends around the corner, so agents may slow near the turn and travel times should include the detour. A successful run ends with the agents reaching the tagged exit.

### V04 — Bottleneck

`V04_Bottleneck.umap`, which is also the current editor startup map, uses `Validation_Bottleneck.xml`. All agents target `BottleExit`.

The narrow section is expected to restrict capacity. Queue length and density should rise near the constriction, and completion should be delayed relative to an open route. The most useful panel values are maximum queue length, peak density, travel time, and throughput.

### V05 — T-junction

`V05_TJunction.umap` uses `Validation_TJunction.xml`. Agent groups are assigned to the left, right, or straight exits, as noted in the schedule comments. This creates branching flow and contention around the junction. Check the destination tag for each group and observe current or peak density at the junction.

### V06 — Functional area

`V06_FunctionalArea.umap` uses `Validation_FunctionalArea.xml`. All agents move from the layout-specific functional zone to `FunctionalExit`. Completion against that tag confirms that agents can leave the area through the intended destination.

### Directional schedule templates

`Validation_MoveLeft.xml` and `Validation_MoveRight.xml` contain template agent blocks for leftward and rightward movement. They are intended for directional spawners and `AScheduleManager::AssignScheduleOverrideToAgent`, rather than as complete multi-agent scenario schedules.

### Running a validation map

Open one of the V01–V06 maps and confirm that its `AScheduleManager` points to the matching file in `Schedules/Validation/`, unless the path is already authored in the map. Build the NavMesh and check that destination actors carry exactly the tags referenced by the XML.

Start Play in Editor with `validationStats/showOnScreenPanel` on and `thesis.ValidationDebugPanel` set to `1`. The panel is live, in-memory only—note numbers down yourself if you need a run-to-run comparison; this path has no CSV export.

### Mall A/B infrastructure verification

The infrastructure comparison belongs to UC3 and does not use a `V0*` validation map. It compares `MallA.umap` and `MallB.umap` using `InfrastructurePlanningSchedule_MallA.xml` and `InfrastructurePlanningSchedule_MallB.xml`.

Each run records infrastructure data such as travel, queues, congestion, stations, paths, popularity, and throughput. With automatic export enabled, the run files follow the pattern `Infrastructure_<Layout>_*.csv`. The comparison produces `VnV_Comparison_MallA_vs_MallB_*.csv` and a text summary named `VnV_Summary_MallA_vs_MallB_*.txt` in `Saved/Analytics`.

The generated checklist covers:

- whether both layout CSV files can be loaded;
- whether travel time was recorded;
- whether queue and station metrics are present;
- whether path utilisation was recorded;
- whether throughput was recorded;
- whether variant waiting time is less than or equal to the baseline;
- whether variant peak congestion is less than or equal to the baseline.

Run both layouts with `autoExportOnEnd` and comparison export enabled. The `comparisonBaseline` and `comparisonVariant` names must match the layout labels in the CSV data. The final CSV and summary files provide the Mall A versus Mall B verification record.

## Debug panel

### Validation Statistics

The Validation Statistics panel is managed by `UValidationDebugPanelSubsystem`. Its values come from the `FValidationAnalyticsSnapshot` maintained by `USimulationAnalytics`.

The panel appears when all three of the following conditions are met:

1. the `ValidationStats` feature is enabled;
2. `showOnScreenPanel` is `true` under `<validationStats>` in `SimulationConfig.xml`;
3. `thesis.ValidationDebugPanel` is set to `1`.

Validation and `V0*_` map names enable the feature through the map-driven gates. The console variable is enabled by default and can be changed while the simulation is running:

```
thesis.ValidationDebugPanel 0
thesis.ValidationDebugPanel 1
```

The same configuration section controls the sample interval and the speed thresholds. Their baseline values are 0.5 seconds per sample, 20 cm/s for waiting, and 80 cm/s for congestion.

### Panel fields

Negative values are sentinels for unsupported or untracked data and are displayed as `n/a`.

| Panel label | Snapshot field | Meaning |
| --- | --- | --- |
| Scenario Name | `ScenarioName` | Normally the loaded map name |
| Simulation Time | `SimulationTimeSeconds` | Elapsed session time in seconds |
| Agents Spawned | `AgentsSpawned` | Agents seen or spawned during the session |
| Agents Active | `AgentsActive` | Agents currently active |
| Agents Completed | `AgentsCompleted` | Agents reported through destination or completion hooks |
| Agents Failed | `AgentsFailed` | Unsupported; `-1` is displayed as `n/a` |
| Average Travel Time | `AverageTravelTimeSeconds` | Mean duration of completed trips |
| Minimum Travel Time | `MinTravelTimeSeconds` | Fastest completed trip |
| Maximum Travel Time | `MaxTravelTimeSeconds` | Slowest completed trip |
| Average Speed | `AverageSpeed` | Mean sampled speed |
| Average Path Length | `AveragePathLength` | Not tracked; `-1` is displayed as `n/a` |
| Throughput (agents/sec) | `ThroughputAgentsPerSecond` | Completions per unit of elapsed time |
| Flow Rate | `FlowRate` | Flow value calculated during analytics sampling |
| Average Queue Length | `AverageQueueLength` | Mean queue value from congestion or station samples |
| Maximum Queue Length | `MaximumQueueLength` | Largest observed queue |
| Peak Density | `PeakDensity` | Highest density sample |
| Current Density | `CurrentDensity` | Most recent density sample |
| Average Waiting Time | `AverageWaitingTimeSeconds` | Mean waiting time |
| Maximum Waiting Time | `MaximumWaitingTimeSeconds` | Longest observed wait |
| Average Congestion Time | `AverageCongestionTimeSeconds` | Mean time spent at or below the congestion-speed threshold |
| FPS | `FPS` | Current frame-rate sample |
| Current Agent Count | `CurrentAgentCount` | Agents counted in the latest sample |
| Current Moving Agents | `CurrentMovingAgents` | Agents above the waiting-speed threshold |
| Current Waiting Agents | `CurrentWaitingAgents` | Agents at or below the waiting-speed threshold |
| Current Average Velocity | `CurrentAverageVelocity` | Mean velocity of the sampled agents |

### Reading a run

Open a V01–V06 validation map and start Play in Editor. The panel is drawn as a cyan on-screen debug block. For a fair comparison, keep spawn counts and other configuration values the same between runs and record the values manually; the validation panel has no CSV export path.

Large maximum queues together with high peak density usually indicate a saturated bottleneck or junction. A high average congestion time means many agents remained below the configured congestion-speed threshold. If throughput is low relative to the spawned count, inspect the level for blocked routes, unresolved destinations, or agents that have not completed. Long travel times with low density more often indicate a long route or detour than crowding.

`n/a` for failed agents and average path length is normal. Those two measurements are not collected.

Mall A/B infrastructure verification uses infrastructure CSV files and the V&V summary rather than this panel.

### Other debug controls

| Control | Effect |
| --- | --- |
| `thesis.AgentAIDebug 0\|1` | Enables periodic priority-layer and agent AI logging from `AgentAIDebug.h` |
| `thesis.VerboseSchedule 0\|1` | Enables detailed schedule, rule, and condition logging |
| `ABoidsAB::DEBUG` | Editor-exposed per-agent debug flag; defaults to `true` |

The `WBP_Evacuation` and `WBP_DailyLife` UMG widgets may display additional run-time information. Their fields are authored in Blueprint rather than enumerated by the C++ panel.

# 6. Configuration

### SimulationConfig.xml

At run time, the feature registry first looks for `Saved/Config/SimulationConfig.xml`. If that file is absent, it loads `Config/SimulationConfig.xml` and copies the baseline into `Saved/Config`.

Use the Saved copy for local experiments. Changes to the Config copy alter the project baseline. In that baseline, Evacuation and Infrastructure Analysis are enabled; the other main features are disabled. Daily Urban Life and Validation Statistics can still be enabled by the map-name gates.
`USimulationFeatureRegistry::ReloadConfig()` reloads the XML from disk.

### Feature flags

The `<features>` section maps XML attributes to `ESimulationFeature` values.

| XML attribute | Feature | Baseline |
|---|---|---|
| `enableEvacuation` | `Evacuation` | `true` |
| `enableLeadership` | `Leadership` | `false` |
| `enableCopsAndRobbers` | `CopsAndRobbers` | `false` |
| `enableInfrastructureAnalysis` | `InfrastructureAnalysis` | `true` |
| `enableDailyUrbanLife` | `DailyUrbanLife` | `false` |
| `enableValidationStats` | `ValidationStats` | `false` |

Daily Urban Life maps, Mall A/B maps, and Validation maps may override these values after configuration loading.

### Daily Urban Life

Settings are read from `<dailyUrbanLife>`.

| XML attribute | C++ setting | Baseline |
|---|---|---|
| `enableNeeds` | `bEnableNeedsDuringDailyLife` | `true` |
| `needsRateScale` | `NeedsRateScale` | `0.1` |
| `startAgentsAtHome` | `bStartAgentsAtHome` | `true` |
| `maxScheduleMinuteOffset` | `MaxScheduleMinuteOffset` | `8` |
| `walkingSpeedMin`, `walkingSpeedMax` | Walking-speed range, cm/s | `170`, `230` |
| `interactionPauseMin`, `interactionPauseMax` | Greeting-pause range, seconds | `2`, `5` |
| `scheduleFile` | `DefaultScheduleFile` | `Schedules/DailyUrbanLifeSchedule.xml` |
| `breakAreaTag` | `BreakAreaTag` | `Plaza` |
| `homeReturnOverridesNeeds` | `bScheduledHomeReturnOverridesNeeds` | `true` |
| `minStationsPerWorkArea` | `MinStationsPerWorkArea` | `4` |

### Evacuation

The `<evacuation>` section controls exit selection, timing, per-agent variation, and optional casualty and celebration states.

| XML attribute | Use | Baseline |
|---|---|---|
| `globalRadius` | Exit-search radius | `999999` |
| `distanceWeight` | Distance term in exit scoring | `1.0` |
| `congestionWeight` | Congestion term in exit scoring | `2.0` |
| `reevaluationInterval` | Seconds between congestion checks | `5.0` |
| `switchThreshold` | Congestion threshold for switching exits | `0.35` |
| `maxReassignmentAttempts` | Maximum number of exit changes | `3` |
| `preferNearestExitInitially` | Applies an initial nearest-exit preference | `true` |
| `reactionDelayMin`, `reactionDelayMax` | Panic reaction delay, seconds | `0`, `1` |
| `evacuationDuration` | Countdown duration; `0` disables the automatic timer | `120` |
| `evacuationTimerStartDelay` | Delay before the countdown starts | `0` |
| `speedVariationMin`, `speedVariationMax` | Per-agent flee-speed multiplier range | `0.85`, `1.15` |
| `congestionToleranceVariation` | Per-agent congestion-threshold variation | `0.15` |
| `nearestExitBiasVariation` | Per-agent nearest-exit bias scale | `0.35` |
| `enableCasualties` | Enables casualty logic | `true` |
| `enableCelebrationState` | Enables the success celebration state | `true` |

The evacuation settings structure also contains `NearestExitBias`, with a C++ default of `0.5`. It is separate from the XML attributes listed above.

### Infrastructure planning

Settings under `<infrastructurePlanning>` control Mall A/B selection, run timing, service capacity, visualisation, and export.

| XML attribute | Use | Baseline |
|---|---|---|
| `autoSelectFromMap` | Selects the Mall A or Mall B schedule and capacity from the map name | `true` |
| `layoutVariant` | Label written for the active layout | `MallA` |
| `comparisonBaseline` | Baseline layout name used by V&V | `MallA` |
| `scheduleFile` | Infrastructure schedule path | Mall A schedule |
| `samplingIntervalSeconds` | Analytics sampling interval | `5` |
| `runDurationSimMinutes` | Run duration in simulated minutes | `10` |
| `enableVisualization` | Enables `AInfrastructureVisualizationActor` use | `true` |
| `reuseDailyUrbanLifeBootstrap` | Reuses tag-based destination setup | `true` |
| `serviceStationCapacity` | Capacity applied to tagged service stations; `0` keeps editor values | `1` |
| `serviceStationTag` | Tag used to select stations for the capacity override | `ServicePoint` |
| `autoExportOnEnd` | Writes the infrastructure CSV when the run ends | `true` |
| `autoExportComparisonOnEnd` | Builds the paired-layout comparison | `true` |
| `comparisonVariant` | Variant layout name | `MallB` |

### Leadership

The `<leadership>` section contains:

| XML attribute | Baseline |
|---|---|
| `defaultFollowDistance` | `200` |
| `formationUpdateInterval` | `0.5` |
| `leadershipOverridesSchedule` | `true` |

### Cops and Robbers

The `<copsAndRobbers>` section contains:

| XML attribute | Baseline |
|---|---|
| `policeDetectionRadius` | `1200` |
| `robberDetectionRadius` | `900` |
| `crimeProbability` | `0.08` |
| `crimeCheckInterval` | `5` |
| `crimeDuration` | `8` |
| `captureRadius` | `150` |
| `pursuitUpdateInterval` | `0.4` |
| `fleeDistance` | `800` |
| `civiliansAvoidRobbers` | `true` |
| `civilianAvoidRadius` | `700` |

### Validation Statistics

The `<validationStats>` settings control the on-screen panel and its classification thresholds.

| XML attribute | Baseline |
|---|---|
| `showOnScreenPanel` | `true` |
| `sampleIntervalSeconds` | `0.5` |
| `waitingSpeedThreshold` | `20` |
| `congestionSpeedThreshold` | `80` |

The speed thresholds are in cm/s.

### AFlockController properties

An `AFlockController` placed in a level supplies shared movement values to the agents that register with it. Level-specific editor values take precedence over constructor defaults.

`MaxSpeed` and `MinSpeed` define the basic speed range. `AlignmentWeight`, `SeparationWeight`, `CohesionWeight`, and `AvoidanceWeight` set the relative strengths of flock alignment, personal spacing, group cohesion, and obstacle or enemy avoidance.

The following properties affect the blend between path following and flock steering during `MoveTo`:

| Property | Role |
| --- | --- |
| `bEnableSteeringWhileMoving` | Allows flock steering while an agent follows a path |
| `NavProgressWeight` | Legacy weight retained for editor compatibility; path progress uses a lateral-only blend basis |
| `MovingSteeringWeight` | Strength of lateral flock steering during movement |
| `AntiProgressDamp` | Legacy editor-compatible property |
| `MaxSteerDeviationDegrees` | Maximum heading deviation from the path |
| `NavMeshSnapDistance` | Distance in centimetres used to snap an off-mesh agent |
| `NavMeshSnapCooldownSeconds` | Minimum time between NavMesh snap attempts |
| `PathLookAheadDistance` | Distance used to look ahead on the current path segment |
| `WallProbeDistance` | Length of the wall probe |
| `WallClearanceDistance` | Preferred side clearance from a wall |
| `WallClearanceStrength` | Strength of the side-clearance correction |

Perception and general steering use `SeparationFOV`, `AlignmentFOV`, and `CohesionFOV` for the three force cones. `NumSonar` and `SonarRange` control obstacle probes. `GoToWeight` controls seeking, and `Acceleration` provides movement acceleration tuning.

Constructor speeds are in the 100–150 range and the default separation and avoidance weights are comparatively high. Placed controller actors may have different values and are authoritative for their map.

### Schedule manager

`AScheduleManager::RelativeSchedulePath` is the editor-facing default XML path. A use-case manager can replace it at run time. The main run-time functions are `SetScheduleFilePath`, `ReloadSchedules`, and `AssignScheduleOverrideToAgent`.

### Stations and areas

`AAgentStation` exposes the following editor properties:

| Property | Role |
| --- | --- |
| `InteractionDuration` | Default time that an agent occupies the station |
| `MaxConcurrentOccupants` | Number of agents admitted at once |
| `StationIdentifier` | Identifier matched against XML `stationType` |
| `InteractionPositionMode` | Selects snap, interpolation, or no position adjustment |
| `InteractionLocationOffset` | Positional offset for the interaction pose |
| `InteractionRotationOffset` | Rotational offset for the interaction pose |
| `InteractionInterpolateSeconds` | Time used to interpolate into the pose |
| `bAlignInteractionRotation` | Aligns the agent with the interaction transform |
| `InteractionAnimType` | Chooses no animation, a montage, or a sequence |
| `InteractionMontage` | Montage used for the interaction |
| `InteractionSequence` | Sequence used for the interaction |
| `bLoopInteractionAnimation` | Loops the selected interaction animation |
| `InteractionAnimSlotName` | Animation slot name |
| `InteractionAnimBlendIn`, `InteractionAnimBlendOut` | Blend times for the animation |

`AAgentArea` groups stations. `bAreaClosed` prevents the area from being used, `DestinationCategory` classifies it as Residential, Business, Public, or None, and `Stations` holds the assigned station actors.

### Navigation

Navigation is configured in the level rather than in `SimulationConfig.xml`. Each navigable map needs a Recast NavMesh. The project has a Recast section in `DefaultEngine.ini`. Sidewalk regions can use `UNavArea_Sidewalk`, and path queries that should remain on those regions use `UNavFilter_SidewalkOnly`.

### Analytics files

| Domain | Output |
| --- | --- |
| Evacuation | `Saved/Analytics/Evacuation_<UnixTimestamp>.csv` |
| Infrastructure | `Saved/Analytics/Infrastructure_<Layout>_<Timestamp>.csv` |
| Mall V&V | `VnV_Comparison_MallA_vs_MallB_<Timestamp>.csv` and `VnV_Summary_MallA_vs_MallB_<Timestamp>.txt` |
| Cops and Robbers | `Saved/Analytics/CopsAndRobbers_<Timestamp>.csv` |
| Daily Urban Life | An export API is available, but the manager does not export automatically |
| Validation | No CSV exporter |

# 7. Core Classes

The simulation code belongs to the `Thesis` module. The classes below form the main run-time path from XML schedules and feature configuration to agent movement, interactions, use-case control, and analytics.

### ABoidsAB

`ABoidsAB` is the pedestrian character and derives from `ACharacter`. It combines local flock steering with navigation, schedule execution, station interactions, evacuation states, and the agent-side AI components.

An agent can compute and follow waypoint paths, start a `MoveTo`, enter a station interaction, flee to an exit or safe zone, and enter evacuation casualty or celebration states. It registers with `AScheduleManager` and `AFlockController`. Feature-dependent work is checked through `USimulationFeatureRegistry`.

The movement path centres on `TargetL`, `StartMoveTo`, `ComputePath`, and `MoveAlongWaypoints`. Station movement uses `MoveToStation`, `TargetStation`, and the interaction pipeline fields. `CurrentBehavior` and the flee and exit helpers track higher-level behaviour. `DEBUG` is exposed for agent debugging.

Each character may host `UAgentNeedsComponent`, `UAgentPrioritySelectorComponent`, `UAgentFallbackHandlerComponent`, `UAgentLeadershipComponent`, `UAgentCrimeRoleComponent`, and `UPedestrianInteractionComponent`.

### AFlockController

`AFlockController` holds the flock membership and shared steering parameters. It does not tick agents on their behalf. Agents register through `AddBoidToTheFlock` and leave through `RemoveBoidFromTheFlock`.

Its getters expose the editor-configured speed limits, alignment, separation, cohesion and avoidance weights, force FOV values, sonar settings, and path/flock movement blend. These values are read by `ABoidsAB` during movement.

### AScheduleManager

`AScheduleManager` owns the simulation clock and executes the XML schedules. It loads agent rules and tasks, registers agents by ID, checks when work is due, evaluates schedule conditions, and sends the resulting commands to `ABoidsAB`.

The supported condition path includes `emergency` and location checks for `Home`, `Work`, and `Food`. Individual agents can receive schedule-file overrides.

`RelativeSchedulePath` is the default editor path. The main API includes `SetScheduleFilePath`, `ReloadSchedules`, `RegisterBoid`, `GetBoidById`, and `AssignScheduleOverrideToAgent`. Daily Urban Life and Infrastructure managers use these functions when they choose a schedule for the current map.

### AAgentStation and AAgentArea

`AAgentStation` is a capacity-limited interaction point with a FIFO queue. `RequestEnter` requests a place, `NotifyOccupantReached` completes arrival at the station, and `LeaveStation` releases the place. `HasAvailableSlot`, `GetQueueLength`, and `GetUtilization` expose its current state. `MatchesIdentifier` compares the station with the identifier requested by the schedule.

The station also supplies getters for its interaction pose and animation settings. Queue and occupancy changes are available through `OnAgentEnqueued` and the admit and release delegates.

`AAgentArea` groups stations and adds an open or closed state and a destination category. Schedule execution uses it to find an available station or a station with a particular identifier for a `UseStation` action.

### Layered agent components

| Class | Responsibility |
|---|---|
| `UAgentNeedsComponent` | Tracks hunger and energy rates and exposes critical-need flags |
| `UAgentPrioritySelectorComponent` | Evaluates priority layers and may issue movement or defer scheduled work |
| `UAgentFallbackHandlerComponent` | Handles stuck agents and path or queue failures |
| `UAgentLeadershipComponent` | Stores leader or follower roles and calculates the follow point |
| `UAgentCrimeRoleComponent` | Runs police, robber, and civilian states |
| `UPedestrianInteractionComponent` | Handles short social pauses and greetings |

### Emergency and world actors

| Class | Responsibility |
|---|---|
| `AEmergencyManager` | Controls the global evacuation lifecycle and timer |
| `AIncidentManager` | Creates incident instances and broadcasts incident changes |
| `AExitPoint` | Provides exit capacity, queue slots, and agent processing |
| `AEvacuationSafeZone` | Marks a safe destination and its arrival radius |
| `UCrowdRoutingService` | Scores evacuation exits; other routing modes use stubs or fallback behaviour |
| `UCongestionAnalyzer` | Samples congestion for route decisions and analytics |

### Use-case managers

`ADailyUrbanLifeManager` starts UC2. It selects the schedule, resolves tagged destinations, assigns walking speeds and schedule offsets, and starts analytics.

`AInfrastructurePlanningManager` runs UC3: layout settings, service-station capacity overrides, run timing, and infrastructure CSV export. `AVerificationValidationManager` reads Mall A and Mall B results and writes the comparison CSV plus text summary. `AInfrastructureVisualizationActor` is the optional UC3 overlay.

`ALeadershipDemoManager` assigns leadership groups when the feature is enabled. `ACopsAndRobbersDemoManager` assigns crime roles and exports crime analytics. `ADirectionalSpawner` creates agents and may give them schedule overrides.

### Simulation subsystems and configuration

| Type | Responsibility |
| --- | --- |
| `USimulationFeatureRegistry` | Stores feature flags and typed settings loaded from XML |
| `USimulationAnalytics` | Samples simulation state and writes supported CSV formats |
| `UValidationDebugPanelSubsystem` | Draws the on-screen validation statistics |
| `FSimulationConfigLoader` | Reads `SimulationConfig.xml` into the feature registry settings |

### Navigation helpers

`UNavArea_Sidewalk` marks sidewalk navigation areas. `UNavFilter_SidewalkOnly` is used for queries that should prefer sidewalk regions and penalise or exclude other areas.

### Analytics data

The selected snapshot structures in `SimulationTypes.h` are:

- `FEvacuationAnalyticsSnapshot`
- `FDailyUrbanLifeAnalyticsSnapshot`
- `FInfrastructureAnalyticsSnapshot`, including station and path entries
- `FCopsAndRobbersAnalyticsSnapshot`
- `FValidationAnalyticsSnapshot`

The same header contains settings structures for evacuation, daily life, infrastructure, leadership, cops and robbers, and validation.

### Legacy and peripheral code

`ARuleParser` is an older B.A.R.S-style parser with hard-coded paths. It is not the active XML schedule path. `APredatorBoid` and `APredatorSpawner` are marked as legacy and are not used by UC1–UC3. `MapProcessing` in Toolbelt is utility and legacy map-XML tooling.

# 8. Extending the Project

Most new scenarios are a map + XML schedule + tags + existing managers. Touch `ABoidsAB` or the schedule executor only when you need a command the current action set cannot express.

### Adding a map

Duplicate the closest existing use-case or validation map so that its world settings and basic actor setup are retained. A scheduled pedestrian map normally needs:

- one `AFlockController`;
- one `AScheduleManager`;
- placed `ABoidsAB` actors or a Blueprint subclass, unless a spawner creates them;
- `AAgentStation` and `AAgentArea` actors where interactions are required;
- the manager actor for the selected use case, such as `AEmergencyManager`, `ADailyUrbanLifeManager`, or `AInfrastructurePlanningManager`.

Add a NavMesh Bounds Volume, build navigation, and inspect the result in the editor. Sidewalk-only routes use `UNavArea_Sidewalk` and `UNavFilter_SidewalkOnly`, so the area assignment must be consistent with the route expected by the agents.

Destination actors need tags that exactly match the XML, for example `Home*`, `Work*`, `Exit*`, or `FoodCourt`. Set `AScheduleManager::RelativeSchedulePath` to the new schedule unless a use-case manager selects it at run time.

Map names participate in feature selection. Existing gates recognise Daily Urban Life names, `MallA` and `MallB`, and names containing `Validation` or the `V0*_` form. A map outside those conventions needs the corresponding feature enabled in `SimulationConfig.xml`, or a new gate added to `ApplyMapDrivenFeatureGates`.

Blueprint defaults and UMG widget setup are binary assets and must be edited in Unreal Editor.

### Writing a schedule

Place schedule files under `Schedules/`; validation schedules are kept under `Schedules/Validation/`. An `<agent id="Agent_N">` entry must use the same ID as the placed or spawned character. Rules, conditions, and `<then>` actions follow the structures used in the existing schedule files and in `04_XML_Guide.md`.

After creating the file, assign it through `AScheduleManager`, or set the relevant use-case manager's `scheduleFile` value. For dynamically spawned agents, a small single-agent template can be assigned with `AssignScheduleOverrideToAgent`.

Action names go through `ParseActionString`. A new string does nothing until you wire the enum/parser and the execution path in `AScheduleManager` / `ABoidsAB`.

When a schedule fails to run, check the agent ID first, followed by the file path, destination tags, station identifiers, and simulation time. `thesis.VerboseSchedule 1` provides rule and condition logs during Play in Editor.

### Adding stations

Place an `AAgentStation` or one of its Blueprint children at each interaction point. Set `StationIdentifier` to the value used by the XML `stationType`. `MaxConcurrentOccupants` controls capacity and `InteractionDuration` controls the default occupation time.

Assign the station to an `AAgentArea`. The area's tag must match the XML `area` name. The area can contain several stations, allowing schedule execution to select a matching available station.

For a posed interaction, choose `InteractionPositionMode` and set the location and rotation offsets. A montage or sequence can be assigned through the animation properties, with looping and blend values where required.

Infrastructure planning can resize service stations at run time. Tag those stations with `ServicePoint`, or with the value configured by `serviceStationTag`. A `serviceStationCapacity` value of `0` leaves the editor capacity unchanged.

Multi-stage interactions use several `<stage>` elements inside a `UseStation` action. Each stage must resolve to a valid area and station identifier.

### Building a validation scenario

Create the level under `Content/UseCases/Validation/` and use a name recognised by the existing gate, such as `V07_Example` or a name containing `Validation`. Place goal actors with unique destination tags, then add a schedule under `Schedules/Validation/` with the required idle and movement rules.

Build the NavMesh, start Play in Editor, and read the Validation Statistics panel. Keep population and configuration values fixed when comparing runs. Validation values are live and in memory, so they must be recorded manually; there is no validation CSV exporter.

Automated assertions would require additional Unreal Automation or Functional Test code, which the project does not currently contain.

### Extending agent behaviour

A new scheduled command requires an `EScheduledAction` entry, parsing in `ParseActionString`, and an execution branch in `AScheduleManager` or `ABoidsAB`. Keep the XML name stable once schedules depend on it.

A new schedule condition belongs in `EvaluateCondition`. If it introduces new state, make sure the manager or agent that owns that state is available when the condition is evaluated.

Priority behaviour is handled by `UAgentPrioritySelectorComponent`. Add a priority enum value and place its evaluation at the intended point in the existing order. Needs belong in `UAgentNeedsComponent`, together with the critical-state handling used by the priority selector.

Stuck, path, or queue recovery should remain in `UAgentFallbackHandlerComponent`. Leadership changes belong in `UAgentLeadershipComponent`, and police, robber, or civilian state changes belong in `UAgentCrimeRoleComponent`.

For a new analytics measurement, add the field to the appropriate snapshot structure and populate it in `USimulationAnalytics`. Add it to the relevant CSV writer only if that use case has an export path.

A new configurable feature needs an `ESimulationFeature` value, an XML attribute handled by `FSimulationConfigLoader`, storage in `USimulationFeatureRegistry`, and registry checks at the call sites. Centralising the check in the registry avoids separate local feature booleans drifting out of sync.

### Adding a use case

Start with the map and identify the actor that will own the run lifecycle. Existing examples are the Daily Urban Life, Infrastructure Planning, Leadership, and Cops and Robbers managers.

If the use case needs tunable values, add a settings structure in `SimulationTypes.h` and load a matching XML section in `FSimulationConfigLoader`. Add an `ESimulationScenario` value when analytics or shared systems need to distinguish the scenario. Automatic selection by map name is added through `ApplyMapDrivenFeatureGates`.

The map must contain the required manager, destinations, schedule manager, flock controller, agents or spawner, and navigation data. Supply a schedule and record every tag or station identifier on which it depends. If the use case records metrics, connect the manager's start and finish points to `USimulationAnalytics` and add an exporter only for the required output format.

### Extending the Mall comparison

Each layout must first produce an infrastructure CSV through `autoExportOnEnd`. The `comparisonBaseline` and `comparisonVariant` values must match the `LayoutVariant` values written into those files. Enable `autoExportComparisonOnEnd`, or use the placed `AVerificationValidationManager`, to generate the comparison CSV and summary.

A new layout name also needs a matching schedule. If it should be selected automatically from the map name, extend the current map-selection logic alongside the Mall A and Mall B cases.

# 9. Known Limitations

Things that are genuinely limited or awkward in the current codebase—not a wish list.

### Content and maps

Binary `.umap` / `.uasset` graphs are not fully visible from source. Actor placement and widget bindings have to be inspected in the editor. There is also no separate requirements PDF in the repo; intent is taken from maps, schedule comments, and `SimulationConfig.xml`.

### Schedules

`MoveTo` picks the **first** actor with the requested tag. Duplicate tags are not load-balanced. Missing tags tend to surface as warnings or fallbacks at runtime, not as a preflight schema check. Rules older than the retry window (default 45 sim minutes) can be skipped as stale. Conditions are only `emergency == true|false` and `location == Home|Work|Food`. Break is a Trigger id (`Break`), not its own action enum value.

### Legacy leftovers

`ARuleParser` still points at hard-coded absolute paths from an older B.A.R.S layout and is not the schedule path used by the use cases. Predator spawner/boid types are unused by UC1–UC3. Some flock fields (for example `AntiProgressDamp`) remain for editor compatibility while MoveTo blend is lateral-only on progress.

### Navigation

If there is no sidewalk-connected route, the sidewalk filter simply fails to produce a usable path. Multi-floor travel needs a continuous mesh (stairs/ramps). Character Movement will not invent vertical travel; gap-closing to a station root does not teleport between floors. Off-mesh XY snap keeps the agent's current Z, which can be wrong if the projected poly is on another floor.

### Stations

Interaction time starts after arrival notification. An empty waypoint list must not start interaction. Pose and animation quality depend on InteractionPoint setup and the AnimBP in content.

### Per use case

**UC2** expects the Home/Work/desk tagging scheme described in the schedule comments. Missing plaza tag falls back to pausing in place for breaks. Daily-life CSV is not auto-exported by the manager.

**UC3** assumes the mall tags referenced in the schedules (shops, food courts, service points, exits, and so on). V&V comparison needs both layout CSVs on disk; checklist lines are not CI gates.

**UC1** is not map-name gated. A map with the feature on but without managers/exits will not evacuate properly. Non-evacuation crowd-routing modes are mostly distance fallbacks.

### Validation

Only V01–V06 maps ship. Name patterns for V07–V09 exist in code but unused. There is no Unreal AutomationTest suite. `AgentsFailed` and average path length show as n/a. There is no validation CSV. Separate "crosswalk / merge / split" maps were never added.

### Defaults that surprise people

Game default map is still `ScheduleTest/Untitled`. Editor startup is a validation bottleneck map. Local config edits belong in `Saved/Config/SimulationConfig.xml` —editing only the project Config copy and wondering why nothing changed is a common footgun.

### Performance

Many agents repathing at once (evac switches, stuck retries) can hitch because pathfinding is synchronous. Dense flocks with high sonar counts are expensive. The validation panel redraws debug text every tick while shown. Fifty UC2 agents sharing desks is intentional queue pressure, but it does load nav and stations harder.

## 10. Conclusion

What exists today is a working Unreal 5.4 pedestrian simulation with flocking, sidewalk-filtered NavMesh paths, XML schedules, and capacity-limited stations. On that base sit three demos: timed evacuation with exit queues and CSV export; a compressed daily-life schedule with homes, desks, needs, and breaks; and a Mall A / Mall B comparison with infrastructure metrics and a short V&V summary when both runs have been exported. Validation maps V01–V06 plus the on-screen statistics panel cover simpler flow experiments.

Config is mostly `SimulationConfig.xml` plus editor properties on `AFlockController` and stations. The limitations chapter lists the sharp edges (legacy RuleParser paths, no validation CSV, map assumptions, and so on).